

第 11 章 图论模型与算法

学习目标

- ☑ 掌握无根树的常用存储法和转化为有根树的方法
- ☑ 掌握由表达式构造表达式树的算法
- ☑ 掌握 Kruskal 算法及其正确性证明，并用并查集实现
- ☑ 掌握基于优先队列的 Dijkstra 算法实现
- ☑ 掌握基于 FIFO 队列的 Bellman-Ford 算法实现
- ☑ 掌握 Floyd 算法和传递闭包的求法
- ☑ 理解最大流问题的概念、流量的 3 个条件、残量网络的概念和求法
- ☑ 理解增广路定理与最小割最大流定理的证明方法，会实现 Edmonds-Karp 算法
- ☑ 理解最小费用最大流问题的概念，以及平行边和反向弧可能造成的问题
- ☑ 会实现基于 Bellman-Ford 的最小费用路算法
- ☑ 学会用网络流算法求解二分图最大基数匹配和最大权完美匹配
- ☑ 学会最小费用循环流的消圈算法

本章介绍一些常见的图论模型和算法，包括最小生成树、单源最短路、每对结点的最短路、最大流、最小费用最大流等。限于篇幅，很多算法都没有给出完整的正确性证明（很容易在其他参考资料中找到相关内容），但给出了简单、易懂的完整代码，方便读者参考。

11.1 再 谈 树

在第 6 章中，我们第一次接触到二叉树；后来，又接触到了其他树状结构，如解答树、BFS 树。本节将继续讨论“树”这一话题。

有 n 个顶点的树具有以下 3 个特点：连通、不含圈、恰好包含 $n-1$ 条边。有意思的是，具备上述 3 个特点中的任意两个，就可以推导出第 3 个，有兴趣的读者不妨试着证明一下。

11.1.1 无根树转有根树

输入一个 n 个结点的无根树的各条边，并指定一个根结点，要求把该树转化为有根树，输出各个结点的父结点编号。 $n \leq 10^6$ ，如图 11-1 所示。

【分析】

树是一种特殊的图，因此很容易想到用

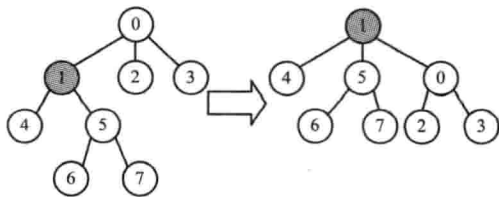


图 11-1 无根树转有根树

邻接矩阵表示。可惜, n 个结点的图对应的邻接矩阵要占用 n^2 个元素的空间, 开不下。怎么办呢? 用 `vector` 数组即可。由于 n 个结点的树只有 $n-1$ 条边, `vector` 数组实际占用的空间与 n 成正比。

```
vector<int> G[maxn];
void read_tree() {
    int u, v;
    scanf("%d", &n);
    for(int i = 0; i < n-1; i++) {
        scanf("%d%d", &u, &v);
        G[u].push_back(v);
        G[v].push_back(u);
    }
}
```

转化过程如下:

```
void dfs(int u, int fa) { //递归转化以 u 为根的子树, u 的父结点为 fa
    int d = G[u].size(); //结点 u 的相邻点个数
    for(int i = 0; i < d; i++) {
        int v = G[u][i]; //结点 u 的第 i 个相邻点 v
        if(v != fa) dfs(v, p[v] = u); //把 v 的父结点设为 u, 然后递归转化以 v 为根的子树
    }
}
```

主程序中设置 `p[root] = -1` (表示根结点的父结点不存在), 然后调用 `dfs(root, -1)` 即可。初学者最容易犯的错误之一就是忘记判断 v 是否和其父结点相等。如果忽略, 将引起无限递归。

11.1.2 表达式树

二叉树是表达式处理的常用工具。例如, $a+b*(c-d)-e*f$ 可以表示成如图 11-2 所示的二叉树。

其中, 每个非叶结点表示一个运算符, 左子树是第一个运算数对应的表达式, 而右子树则是第二个运算数对应的表达式。如何给一个表达式建立表达式树呢? 方法有很多, 这里只介绍一种: 找到“最后计算”的运算符 (它是整棵表达式树的根), 然后递归处理。下面是程序:

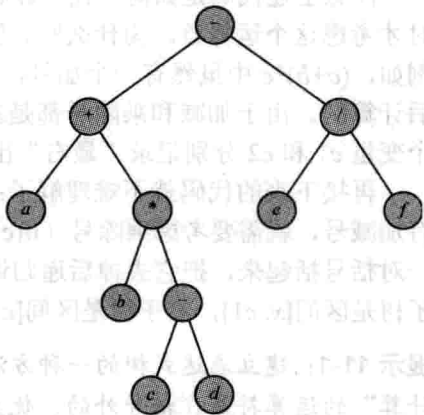


图 11-2 表达式树

```
const int maxn = 1000;
int lch[maxn], rch[maxn]; char op[maxn]; //每个结点的左右子结点编号和字符
int nc = 0; //结点数
```

```
int build_tree(char* s, int x, int y) {
    int i, c1=-1, c2=-1, p=0;
    int u;
    if(y-x == 1){ //仅一个字符，建立单独结点
        u = ++nc;
        lch[u] = rch[u] = 0; op[u] = s[x];
        return u;
    }
    for(i = x; i < y; i++) {
        switch(s[i]) {
            case '(': p++; break;
            case ')': p--; break;
            case '+': case '-': if(!p) c1=i; break;
            case '*': case '/': if(!p) c2=i; break;
        }
    }
    if(c1 < 0) c1 = c2; //找不到括号外的加减号，就用乘除号
    if(c1 < 0) return build_tree(s, x+1, y-1); //整个表达式被一对括号括起来
    u = ++nc;
    lch[u] = build_tree(s, x, c1);
    rch[u] = build_tree(s, c1+1, y);
    op[u] = s[c1];
    return u;
}
```

注意上述代码是如何寻找“最后一个运算符”的。代码里用了一个变量 p ，只有当 $p=0$ 时才考虑这个运算符。为什么呢？因为括号里的运算符一定不是最后计算的，应当忽略。例如， $(a+b)*c$ 中虽然有一个加号，但却是在括号里的，实际上比它优先级高的乘号才是最后计算的。由于加减和乘除号都是左结合的，最后一个运算符才是最后计算的，所以用两个变量 $c1$ 和 $c2$ 分别记录“最右”出现的加减号和乘除号。

再接下来的代码就不难理解了：如果括号外有加减号，它们肯定最后计算；但如果没加减号，就需要考虑乘除号（if($c1 < 0$) $c1 = c2$ ）；如果全都没有，说明整个表达式外面被一对括号括起来，把它去掉后递归调用。这样，就找到了最后计算的运算符 $s[c1]$ ，它的左子树是区间 $[x, c1]$ ，右子树是区间 $[c1+1, y]$ 。

提示 11-1：建立表达式树的一种方法是每次找到最后计算的运算符，然后递归建树。“最后计算”的运算符是在括号外的、优先级最低的运算符。如果有多个，根据结合性来选择：左结合的（如加、减、乘、除）选最右边；右结合的（如乘方）选最左边。根据规定，优先级相同的运算符的结合性总是相同。

例题 11-1 公共表达式消除 (Common Subexpression Elimination, ACM/ICPC NWERC 2009, UVa12219)

可以用表达式树来表示一个表达式。在本题中，运算符均为二元的，且运算符和运算